

Comparative Analysis of Agent Orchestration Frameworks for AWS and Databricks Environments

1. Introduction: The Evolving Landscape of Agent Orchestration in Hybrid Cloud Environments

The increasing sophistication and prevalence of artificial intelligence and machine learning applications have led to the emergence of complex workflows involving autonomous and collaborative AI agents. These agents, designed to perform specific tasks and interact with their environment, necessitate robust orchestration frameworks to manage their lifecycles, communication, and the allocation of computational resources. Modern AI applications frequently involve intricate interactions between multiple specialized agents to achieve complex objectives, such as automated customer service, content generation, and sophisticated data analysis. The effective management of these multi-agent systems is paramount for ensuring the reliability, efficiency, and scalability of AI solutions.

Hybrid cloud environments, exemplified by the combined use of Amazon Web Services (AWS) and Databricks, present both unique challenges and significant opportunities for the deployment and management of AI agents. Organizations often leverage the broad spectrum of services and the mature ecosystem offered by AWS for various aspects of their infrastructure and application deployment. Simultaneously, Databricks is frequently chosen for its unified data analytics platform, which is highly optimized for big data processing and machine learning workflows. Orchestrating AI agents seamlessly across these diverse environments requires frameworks that possess strong integration capabilities with the specific tools and services of each platform, along with flexible deployment options that can accommodate the distributed nature of hybrid cloud architectures.

The field of agent orchestration is closely related to and often builds upon the foundations of data orchestration. Many concepts integral to data orchestration, such as the programmatic definition of workflows, the scheduling of tasks, and comprehensive monitoring, are equally crucial in the context of managing AI agents¹. It is observable that tools initially conceived for orchestrating data pipelines are now being adapted or are offering new features that are directly relevant to the orchestration of AI agents. This evolution suggests that a comprehensive understanding of the landscape requires considering frameworks that have either originated from or have significantly expanded into the domain of AI agent orchestration. The existence of a multitude of agent orchestration frameworks

underscores the dynamism and the rapid evolution of this field ². This proliferation of options reflects the growing recognition of the importance of agent orchestration in AI development and the diverse strategies being explored to address the inherent complexities of managing autonomous AI entities. Consequently, the selection of the most appropriate framework is often highly dependent on the specific use cases, technical requirements, and the existing infrastructure of an organization.

2. Defining Key Evaluation Criteria for Agent Orchestration Frameworks

To effectively compare and evaluate agent orchestration frameworks for deployment across AWS and Databricks, several key criteria must be considered:

- **Scalability:** This refers to the framework's capacity to efficiently manage a growing number of agents and increasing workloads. It includes the mechanisms for both horizontal scaling (adding more agents or instances) and vertical scaling (increasing the resources of existing agents or instances). A scalable framework should be able to handle concurrent tasks, adapt to fluctuating computational demands, and maintain performance levels as the system expands.
- **Ease of Use:** This encompasses the learning curve for developers adopting the framework, the overall quality of the development experience, and the availability of user-friendly interfaces. This includes the intuitiveness of the framework's Application Programming Interface (API), the clarity and completeness of its documentation, and the presence of tools such as Command Line Interfaces (CLIs), graphical User Interfaces (GUIs), and Software Development Kits (SDKs) that streamline the development, debugging, and deployment processes.
- **Integration Capabilities (AWS & Databricks):** This criterion assesses how well the framework can interact with common and essential tools and services within both the AWS ecosystem (e.g., Elastic Compute Cloud (EC2), Simple Storage Service (S3), Identity and Access Management (IAM), Bedrock, SageMaker) and the Databricks platform (e.g., clusters, notebooks, Model Serving, Vector Search, Unity Catalog). Seamless integration is vital for leveraging the specific strengths of each platform and for constructing end-to-end AI applications that can effectively utilize the available cloud resources and services.
- **Monitoring:** This refers to the built-in or readily available capabilities for tracking the performance of agents, their resource utilization (e.g., CPU, memory), and any potential issues that may arise during operation. This includes features such as logging, tracing of agent activities, and the collection of performance metrics. Robust monitoring is essential for gaining insights into agent behavior, diagnosing problems, and ensuring the overall reliability and quality of applications powered

by these agents.

- **Local Development Options:** This criterion evaluates the availability of methods and resources that allow developers to experiment with the framework on their local machines without requiring immediate access to AWS or Databricks cloud services. This can include the provision of Docker images, comprehensive tutorials, or specific instructions for setting up a local development environment (e.g., using tools like Minikube or similar). The availability of such options is crucial for facilitating rapid prototyping, enabling experimentation, and lowering the initial barriers to entry for developers who may not yet have access to or want to immediately engage with cloud-based services.

3. Comparative Analysis of Top Agent Orchestration Frameworks

3.1. Framework 1: LangChain

Overview and Architecture: LangChain is a widely adopted framework designed to simplify the development of applications powered by large language models (LLMs) ³. Its architecture is inherently modular and highly extensible, providing developers with a comprehensive collection of components and integrations that can be combined to construct sophisticated AI applications, including those involving both single and multiple interacting agents ³. This versatility has contributed to its popularity within the AI/ML community ³.

Scalability Considerations and Mechanisms: For building scalable agent orchestration, LangChain offers LangGraph, a framework specifically designed for creating stateful, multi-actor applications that leverage LLMs ⁹. LangGraph allows developers to model agent workflows as directed graphs, where nodes can represent individual agents or specific actions, and edges define the flow of information or control between these nodes. This graph-based approach is particularly well-suited for constructing complex, multi-agent systems capable of handling intricate tasks. The scalability of such systems can be achieved by adding more nodes to the graph to represent additional agents or parallelizing the execution of independent branches within the workflow. LangGraph also incorporates features like persistence and streaming, which are crucial for building production-grade agents that can maintain state across interactions and provide real-time feedback. Notably, LangGraph powers production agents used by companies such as LinkedIn, Uber, Klarna, and GitLab, indicating its capability to handle real-world scale ⁹.

Ease of Use: LangChain aims to simplify the development process for LLM-powered applications, including agent orchestration, by providing a rich set of open-source components and numerous third-party integrations ⁹. LangGraph further enhances

this by offering abstractions specifically tailored for building stateful agents, thereby simplifying the development of more advanced agentic workflows ⁹. While the breadth and depth of LangChain's capabilities can initially present a steeper learning curve for new users, its modular design and the extensive documentation available help in managing this complexity. The framework's well-structured documentation, including conceptual guides, tutorials, and API references, provides comprehensive support for developers at various levels of expertise ⁷.

Integration with AWS and Databricks: LangChain provides dedicated integration packages for both AWS (langchain-aws) and Databricks (langchain-databricks), facilitating seamless interaction with services from both platforms ⁴. Specifically, LangChain integrates with AWS Bedrock, allowing agents to access and utilize foundation models offered by AWS ¹⁴. Within the Databricks ecosystem, LangChain offers strong support and specific integrations with services such as Databricks Model Serving and Vector Search ⁴. These integrations enable LangChain agents to leverage the scalable compute, data storage, and vector database capabilities provided by both AWS and Databricks. Databricks itself provides a wide array of LangChain integrations through the databricks-langchain partner package, which includes first-party integrations for various Databricks services, thereby streamlining the development of LangChain applications within the Databricks environment ⁵. This robust integration landscape underscores the suitability of LangChain for orchestrating agents in hybrid environments involving both AWS and Databricks.

Monitoring Tools and Capabilities: LangChain offers monitoring capabilities through its LangSmith platform, which is designed for tracing and evaluating language model applications and intelligent agents ⁸. LangSmith provides developers with the necessary tools to inspect the behavior of their agents, debug any issues that may arise, and evaluate their overall performance, which is crucial for transitioning applications from the prototyping phase to production deployment. Furthermore, when LangChain is used within the Databricks environment, it benefits from integration with MLflow, a platform for tracking machine learning experiments and managing the lifecycle of models ⁴. MLflow on Databricks offers additional features that enhance the development experience, such as a fully managed MLflow Tracking Server and seamless integration with Databricks Notebooks ⁴. This integration allows users to effectively monitor their LangChain experiments and deployments within the Databricks platform, providing valuable insights into the performance and behavior of their agent orchestration systems.

Local Development Options and Resources: LangGraph is a pivotal tool for enabling local development of agent orchestration workflows within the LangChain

ecosystem⁹. The graph-based nature of LangGraph allows developers to define and test the interactions between agents on their local machines before deploying them to cloud environments. The documentation and tutorials specifically for LangGraph likely contain detailed information and guidance on setting up and utilizing local development workflows⁹. Additionally, LangChain itself, being a Python-based framework, can be readily developed and tested using standard Python development environments and tools. This facilitates rapid prototyping and experimentation without the immediate need for cloud resources, allowing developers to iterate and refine their agent orchestration logic locally.

Strengths: LangChain boasts broad integrations with a vast array of LLMs, external tools, and platforms, offering exceptional flexibility in building AI applications. It benefits from a large and active community that provides extensive support, resources, and continuous contributions to the framework¹⁸. The design of LangChain is highly modular and extensible, allowing developers to customize and tailor solutions to their specific needs. Furthermore, it offers strong support and specific integrations within the Databricks ecosystem, making it a powerful choice for users leveraging that platform.

Weaknesses: The sheer number of features and available integrations within LangChain can lead to a steeper initial learning curve for developers new to the framework. Managing dependencies in complex LangChain projects, especially those involving numerous external integrations and large language models, can sometimes be challenging³. Additionally, running LangChain applications, particularly those that heavily utilize large language models and external APIs, can be resource-intensive.

3.2. Framework 2: Databricks Mosaic AI Agent Framework

Overview and Integration within Databricks: The Databricks Mosaic AI Agent Framework is a suite of tools tightly integrated within the Databricks platform, designed to facilitate the building of production-scale agent systems using any large language model²¹. The framework places a strong emphasis on Retrieval Augmented Generation (RAG) applications, providing comprehensive tooling for their development and deployment²¹. By leveraging the Databricks Data Intelligence Platform, Mosaic AI offers a unified environment for building, deploying, evaluating, and managing generative AI applications, ensuring security and cost-efficiency³⁰.

Scalability Features and Performance: The Mosaic AI Agent Framework is engineered to support the development and deployment of high-quality generative AI applications at scale²¹. Its deep integration with the Databricks infrastructure allows it to leverage the platform's inherent scalability in both compute and data management.

This enables the framework to efficiently handle a large number of agents and high computational workloads without significant performance degradation. Features like automated real-time pipelines for various data types further contribute to its ability to manage increasing workloads effectively ²¹.

Ease of Use: The framework is designed to streamline the process of evaluating and iterating on RAG applications, simplifying feedback collection and evaluation for developers ²¹. It offers a low-code user interface through the AI Playground, which allows for rapid prototyping of agents ²⁵. This tool enables users to select from various LLMs and quickly add tools using a visual interface, making it accessible even to those with limited coding experience. The framework's tight integration with the Databricks environment also simplifies development for users who are already familiar with the platform's ecosystem and tools.

Integration with AWS and Databricks: The Mosaic AI Agent Framework exhibits deep integration with various Databricks services, including Model Serving, Vector Search, and Unity Catalog ²¹. These integrations allow agents to seamlessly access and utilize Databricks' capabilities for model deployment, semantic search, and unified data governance. While the framework primarily focuses on leveraging the Databricks platform, it can also be used in conjunction with AWS services like Bedrock through integration with frameworks such as LangChain ¹⁴. However, direct and deep integrations with other AWS services beyond the underlying infrastructure are not explicitly detailed. The framework's strength lies in its optimized performance and tight integration within the Databricks environment, making it a natural choice for organizations heavily invested in this platform.

Monitoring and Evaluation Capabilities: The Mosaic AI Agent Framework provides built-in AI-assisted evaluation tools and an intuitive user interface to facilitate continuous improvement of agent applications ²¹. It is also integrated with Lakehouse Monitoring, offering a unified solution for ensuring the quality of both data and AI applications ²¹. MLflow is utilized for logging and tracing agent executions ²³, providing detailed insights into agent behavior, performance metrics, and enabling effective debugging. The Agent Evaluation Review App further enhances monitoring by allowing domain experts to assess, label, and define criteria for agent performance without the need for external tools like spreadsheets ²¹.

Local Development Options and Resources: The framework provides specific guidance on the local development of Vector Search retriever tools using Databricks AI Bridge ²¹. This allows developers to prototype and test retrieval functionalities locally before deploying them as part of a Databricks-based agent. Additionally, the AI

Playground offers a low-code environment for prototyping agents, which can then be exported to code for further development and deployment on the Databricks platform²⁵. While the framework is deeply connected to the Databricks platform, these local development options allow for initial experimentation and component-level development without requiring immediate cloud access.

Strengths: The Databricks Mosaic AI Agent Framework offers seamless and deep integration within the Databricks ecosystem, providing a cohesive and optimized experience for users of the platform. It places a strong emphasis on data governance and security through its integration with Unity Catalog. The framework provides comprehensive tooling that covers the entire lifecycle of AI agent development, including robust evaluation and monitoring capabilities. The availability of a low-code option for prototyping through the AI Playground lowers the barrier to entry for some users.

Weaknesses: The framework is primarily focused on the Databricks platform, with less emphasis on direct and deep integrations with other cloud providers like AWS, beyond the fact that Databricks itself can run on AWS infrastructure. Local development options may be more limited to specific components of the agent rather than the entire framework, reflecting its tight coupling with the Databricks environment.

3.3. Framework 3: CrewAI

Overview and Focus on Multi-Agent Systems: CrewAI is an open-source Python framework specifically designed for developing and managing multi-agent AI systems that can collaborate to tackle complex tasks³¹. Its core concept involves the creation of "crews," which are teams of specialized AI agents that work together towards a common objective, closely mirroring the structure of human teams within an organization³². This specialization makes CrewAI particularly suitable for applications that require intricate collaboration and coordination among multiple autonomous AI entities.

Scalability in Managing Collaborative Agents: CrewAI is designed to scale effectively, ensuring that multi-agent systems remain responsive and efficient as they grow in complexity and size³². The framework's architecture supports the creation of crews with varying numbers of agents, and it provides mechanisms for managing their interactions, task distribution, and resource allocation as the scale of the system increases. The ability to define specific roles and responsibilities for each agent within a crew allows for a modular and well-structured approach to building scalable

multi-agent solutions ³².

Ease of Use: CrewAI aims to provide a platform that simplifies the process of building multi-agent automations, even for users without extensive coding experience, through its framework and potentially a UI Studio ⁴⁹. The platform also includes a simple management UI for overseeing AI agents, facilitating human-in-the-loop feedback and control ⁴⁹. CrewAI offers a high-level API that abstracts away much of the underlying complexity involved in creating and managing agents and their tasks, allowing developers to focus on defining the roles, goals, and collaborative interactions of their AI teams.

Integration with AWS and Databricks: CrewAI offers good integration with both the AWS and Databricks ecosystems ³³. It integrates with AWS services such as SageMaker for hosting large language models and Bedrock for accessing foundation models ³³. For Databricks, CrewAI provides direct integration with Databricks Unity Catalog, enabling users to define and utilize Databricks functions as tools for their agents ³⁸. Additionally, CrewAI leverages LiteLLM, which provides broad compatibility with various language model providers, including those available on Databricks Model Serving ³⁷. This cross-platform integration capability makes CrewAI a versatile option for orchestrating agents in hybrid cloud environments.

Monitoring Features and Tools: CrewAI provides monitoring capabilities that allow users to track the performance and progress of their multi-agent crews on both simple and complex tasks ³³. When deployed on AWS, CrewAI can work in conjunction with Amazon CloudWatch for more comprehensive monitoring of the underlying infrastructure and application logs ³³. The framework offers features for observing the execution of tasks and the interactions between agents, providing valuable insights into the operational dynamics of the multi-agent system.

Local Development Possibilities: CrewAI offers a self-hosted option, which allows users to deploy the platform on their local infrastructure, providing complete control over their development environment ⁴⁹. Being a Python-based framework, CrewAI can be easily installed and run locally, enabling developers to build, test, and experiment with multi-agent systems on their local machines before considering deployment to cloud environments like AWS or Databricks.

Strengths: CrewAI excels in facilitating the creation of complex multi-agent systems with a strong emphasis on collaborative problem-solving. It offers good integration with both the AWS and Databricks ecosystems through LiteLLM and specific integrations like Unity Catalog. The availability of a self-hosted option supports local

development and deployment flexibility. CrewAI is also noted for its relatively user-friendly high-level API, which simplifies the development of sophisticated agent interactions.

Weaknesses: While CrewAI integrates well with both AWS and Databricks, the depth of integration with specific services within each platform might not be as profound as that offered by platform-native frameworks. The level of abstraction provided by its high-level API, while beneficial for ease of use, might limit the ability to exert fine-grained control over certain aspects of agent behavior and infrastructure in highly specialized scenarios.

3.4. (Optional) Framework 4: AutoGen

Overview and Focus: AutoGen, developed by Microsoft, is a framework designed to automate the generation of code, models, and processes necessary for complex workflows, with a particular emphasis on simplifying the creation of AI agents ³. A key strength of AutoGen is its ability to facilitate the creation of multiple agents that can work together collaboratively to solve tasks, often involving intricate conversations and the use of various tools ¹⁶. AutoGen aims to make AI development more accessible by automating many aspects of agent creation and workflow definition ³.

Scalability: AutoGen is architected to support the creation and management of multiple interacting agents, making it inherently suitable for building scalable multi-agent systems ¹⁶. The framework allows developers to define different types of agents with specific capabilities and then orchestrate their interactions to achieve complex goals, suggesting that scaling can be achieved by adding more agents or configuring existing agents to handle larger workloads.

Ease of Use: AutoGen is designed with a user-friendly approach, making it accessible even to developers who may not have an extensive background in AI ³. The framework provides a relatively straightforward API for defining agents and their interactions, which can lower the barrier to entry for those looking to build multi-agent applications. Its focus on automation further contributes to ease of use by streamlining many of the traditionally manual steps involved in agent development.

Integration with AWS and Databricks: AutoGen offers tracing integrations for AWS Bedrock, allowing for the monitoring of agent behavior when using foundation models from AWS ¹⁶. While the provided research snippets do not explicitly mention deep integration with Databricks services, AutoGen, being a Python-based framework, could potentially interact with Databricks through its general integration capabilities

or by leveraging other Python libraries that provide connectivity to Databricks.

Monitoring: AutoGen has tracing capabilities through its integration with platforms like Arize, enabling the monitoring of the behavior and performance of AutoGen agents ¹⁶. This is crucial for debugging issues, optimizing performance, and gaining insights into how the agents are functioning within the multi-agent system.

Local Development Options: Given its focus on automation and its Python-based nature, local development with AutoGen is feasible for building and testing agents before their deployment. Developers can install and run AutoGen locally using standard Python development environments, allowing them to experiment with various multi-agent workflows and configurations on their local machines.

Strengths: AutoGen excels in automating the creation and collaborative interaction of multiple AI agents. It features a user-friendly design and API, making it accessible to a wider range of developers. The framework also offers integration with the Microsoft ecosystem and provides tracing capabilities for AWS Bedrock.

Weaknesses: Compared to more comprehensive frameworks like LangChain, AutoGen might prioritize standardization over the ability to deeply customize every aspect of the agent architecture ³. Additionally, the provided research snippets offer limited explicit details regarding deep integration with specific Databricks services.

3.5. (Optional) Framework 5: AWS Multi-Agent Orchestrator

Overview and Focus: The AWS Multi-Agent Orchestrator is a framework provided by AWS for managing multiple AI agents and facilitating complex, multi-turn conversations ⁵³. As a managed service within the AWS ecosystem, it is designed to simplify the process of building, deploying, and managing multi-agent systems on the AWS platform ². This framework aims to provide developers with pre-built components and tools that accelerate the development and deployment of sophisticated agent-based applications.

Scalability: The AWS Multi-Agent Orchestrator is designed for seamless coordination and leveraging multiple AI agents within a single system ⁵³. Being a managed service on AWS, it benefits from the underlying scalability and reliability of the AWS infrastructure. This allows it to handle a growing number of agents and increasingly complex workloads, automatically scaling resources as needed to maintain performance.

Ease of Use: The framework offers pre-built components that are intended to enable

quick development and deployment of multi-agent systems⁵⁴. This suggests that AWS has focused on providing a streamlined development experience by offering ready-to-use modules and abstractions that simplify the creation and management of complex agent interactions and workflows.

Integration with AWS and Databricks: As an integral part of AWS, the Multi-Agent Orchestrator has seamless integration with other AWS services, including Bedrock². This allows agents within the framework to easily access and utilize the foundation models and other AI capabilities offered by AWS. While the framework operates on AWS infrastructure, the provided research snippets do not explicitly detail deep or direct integration with Databricks services. However, it is conceivable that agents within this framework could interact with Databricks through general API calls or other integration mechanisms if required.

Monitoring: The framework provides features such as intelligent intent classification and context management, which are essential for monitoring and managing complex conversations involving multiple agents⁵³. As an AWS service, it likely integrates with AWS monitoring tools like CloudWatch, although this is not explicitly confirmed in the snippets. These features would allow developers to track the flow of conversations, understand how user intents are being handled, and manage the context of interactions across multiple agents.

Local Development Options: The AWS Multi-Agent Orchestrator supports universal deployment, which includes the ability to run the framework in local development environments⁵³. This allows developers to build and test their multi-agent systems locally before deploying them to the full AWS environment. The availability of example projects in Python further facilitates local experimentation and development⁵⁴.

Strengths: The AWS Multi-Agent Orchestrator offers seamless and deep integration with the broad AWS ecosystem, making it a compelling choice for organizations heavily invested in AWS. It supports various deployment options, including local development, and provides pre-built components that contribute to ease of use and rapid development.

Weaknesses: The primary focus of this framework is on the AWS platform, and the provided research snippets do not indicate deep or direct integration with Databricks services. As a relatively newer framework compared to more established options like LangChain, the community support and the breadth of available resources might currently be more limited.

4. Comparative Summary and Recommendations

4.1. Table: Comparison of Agent Orchestration Frameworks

Feature	LangChain	Databricks Mosaic AI Agent Framework	CrewAI	AutoGen (Optional)	AWS Multi-Agent Orchestrator (Optional)
Scalability	Good (via LangGraph)	Excellent (Databricks infrastructure)	Good	Good	Excellent (AWS infrastructure)
Ease of Use	Moderate	Good	Good	Good	Good
AWS Integration	Extensive (inc. Bedrock, SageMaker)	Limited direct, works on AWS	Good (inc. SageMaker, Bedrock via LiteLLM)	Good (Bedrock tracing)	Excellent
Databricks Integration	Excellent (Model Serving, Vector Search)	Excellent (deep platform integration)	Good (Unity Catalog, Model Serving via LiteLLM)	Limited explicit mention	Limited explicit mention
Monitoring	Good (LangSmith, MLflow)	Excellent (Databricks monitoring, MLflow)	Good (CrewAI monitoring, CloudWatch on AWS)	Good (Arize tracing)	Good (intent classification , context)
Local Development	Good (LangGraph, standard Python)	Moderate (AI Bridge for components)	Good (self-hosted option)	Good (standard Python)	Good

4.2. Discussion

The analysis reveals that LangChain offers a high degree of flexibility and a vast ecosystem of integrations, making it a versatile choice for orchestrating agents across different platforms. Its strength lies in its ability to connect with a wide range of LLMs and tools, although this extensive functionality can also lead to increased complexity. The Databricks Mosaic AI Agent Framework stands out for its tight integration with the Databricks platform, excelling in scenarios focused on RAG and data-centric AI applications. It provides robust governance and monitoring capabilities within the Databricks environment, making it ideal for users heavily invested in this platform. CrewAI is particularly well-suited for orchestrating complex collaborations between multiple agents, offering good cross-platform support for both AWS and Databricks through LiteLLM and specific integrations. Its focus on ease of use for building team-like agent structures makes it accessible for developing sophisticated multi-agent systems. AutoGen provides a unique approach by focusing on automating the creation and management of multiple agents for collaborative tasks, with a user-friendly design. The AWS Multi-Agent Orchestrator offers the most seamless experience for users who are primarily working within the AWS ecosystem, providing a managed service that simplifies multi-agent orchestration on AWS infrastructure.

4.3. Recommendations

Based on the comparative analysis, the following recommendations can be made:

- For users requiring maximum flexibility and broad integrations across a diverse range of LLMs, tools, and platforms, including strong support for both AWS and Databricks, **LangChain** is a highly recommended framework. Its extensive ecosystem and active community provide ample resources and support for building complex agent orchestration solutions.
- Organizations that are heavily invested in the Databricks ecosystem and are primarily focused on building Retrieval Augmented Generation (RAG) applications with strong emphasis on data governance and security should strongly consider the **Databricks Mosaic AI Agent Framework**. Its deep integration with Databricks services offers an optimized and cohesive development and deployment experience.
- If the primary focus is on orchestrating intricate collaborations between multiple autonomous AI agents, with a need for good cross-platform support for both AWS and Databricks, **CrewAI** is an excellent choice. Its high-level API and focus on creating team-like agent structures simplify the development of complex multi-agent systems.
- For teams looking for a framework that automates the creation and management of multiple agents and offers a user-friendly approach to building collaborative AI

workflows, **AutoGen** is a worthwhile option to explore.

- Organizations that are primarily operating within the AWS ecosystem and are seeking a managed service to simplify the orchestration of multiple AI agents should consider the **AWS Multi-Agent Orchestrator**. Its seamless integration with other AWS services provides a streamlined experience for building and deploying agent-based applications on AWS infrastructure.

5. Conclusion

The comparative analysis of agent orchestration frameworks reveals that LangChain, the Databricks Mosaic AI Agent Framework, and CrewAI are among the top contenders for building, deploying, and managing multiple agents across hybrid AWS and Databricks environments. Each framework presents a distinct set of strengths and trade-offs, making the optimal choice contingent upon the specific requirements and priorities of the project.

Emerging trends in the field of agent orchestration indicate an increasing focus on multi-agent systems as a means to tackle more complex and nuanced problems. There is also a growing emphasis on the development of sophisticated monitoring and evaluation tools to ensure the reliability, performance, and overall quality of agent-based applications. Furthermore, the increasing support for hybrid and multi-cloud deployments reflects a broader industry trend towards providing greater flexibility and choice in infrastructure. The field of agent orchestration is expected to continue its rapid evolution, with new frameworks, features, and best practices emerging to meet the growing demand for robust and scalable solutions capable of managing the next generation of AI applications.

Works cited

1. Top 17 Data Orchestration Tools for 2025: Ultimate Review - lakeFS, accessed on March 18, 2025, <https://lakefs.io/blog/data-orchestration-tools-2023/>
2. List of Top 10 Multi-Agent Orchestrator Frameworks for Deploying AI Agents, accessed on March 18, 2025, <https://www.devopsschool.com/blog/list-of-top-10-multi-agent-orchestrator-frameworks-for-deploying-ai-agents/>
3. Top 9 AI Agent Frameworks as of February 2025 - Shakudo, accessed on March 18, 2025, <https://www.shakudo.io/blog/top-9-ai-agent-frameworks>
4. LangChain on Databricks for LLM development, accessed on March 18, 2025, <https://docs.databricks.com/aws/en/large-language-models/langchain>
5. Databricks - LangChain, accessed on March 18, 2025, <https://python.langchain.com/docs/integrations/providers/databricks/>
6. LangChain on Databricks for LLM development, accessed on March 18, 2025,

- <https://docs.gcp.databricks.com/en/large-language-models/langchain.html>
7. Introduction |  Langchain, accessed on March 18, 2025, <https://js.langchain.com/docs/introduction>
 8. langchain-ai/langchain: Build context-aware reasoning applications - GitHub, accessed on March 18, 2025, <https://github.com/langchain-ai/langchain>
 9. Introduction |  LangChain, accessed on March 18, 2025, <https://python.langchain.com/>
 10. LangGraph - LangChain, accessed on March 18, 2025, <https://www.langchain.com/langgraph>
 11. LangGraph - Conceptual Guide - GitHub Pages, accessed on March 18, 2025, <https://langchain-ai.github.io/langgraph/concepts/>
 12. langgraph/docs/docs/concepts/low_level.md at main - GitHub, accessed on March 18, 2025, https://github.com/langchain-ai/langgraph/blob/main/docs/docs/concepts/low_level.md
 13. LangGraph - LangChain Blog, accessed on March 18, 2025, <https://blog.langchain.dev/langgraph/>
 14. How to build a RAG with AWS Bedrock and Databricks | by Sarbani Maiti | Medium, accessed on March 18, 2025, <https://medium.com/@sarbaniitb2020/how-to-build-a-rag-with-aws-bedrock-and-databricks-b557fa6bba09>
 15. B Building an Intelligent Agent for Query Conversion: AWS Glue Catalog to Unity Catalog in Databricks | by THE BRICK LEARNING | Jan, 2025 | Medium, accessed on March 18, 2025, <https://medium.com/@infinitylearnings1201/building-an-intelligent-agent-for-query-conversion-aws-glue-catalog-to-unity-catalog-in-databricks-856b8bf0d630>
 16. Autogen | Arize Docs, accessed on March 18, 2025, <https://docs.arize.com/arize/llm-tracing/tracing-integrations-auto/autogen>
 17. Orchestrating an AI Agent Prototype with Hex, Dagster, and LangChain - RepublicOfData.io, accessed on March 18, 2025, <https://blog.republicofdata.io/orchestrating-an-ai-agent-prototype-with-hex-dagster-and-langchain-2/>
 18. The LangChain Community, accessed on March 18, 2025, <https://www.langchain.com/community>
 19. Join the LangChain Community, accessed on March 18, 2025, <https://www.langchain.com/join-community>
 20. Langchain - DEV Community, accessed on March 18, 2025, <https://dev.to/t/langchain>
 21. Mosaic AI Agent Framework - Databricks, accessed on March 18, 2025, <https://www.databricks.com/product/machine-learning/retrieval-augmented-generation>
 22. Build and deploy quality AI agent systems - Databricks, accessed on March 18, 2025, <https://www.databricks.com/product/artificial-intelligence>
 23. Build an Autonomous AI Assistant with Mosaic AI Agent Framework | Databricks Blog, accessed on March 18, 2025,

- <https://www.databricks.com/blog/build-autonomous-ai-assistant-mosaic-ai-agent-framework>
24. Databricks Unveils Mosaic AI Agent Framework - Linqto, accessed on March 18, 2025,
<https://www.linqto.com/unicorn-news/databricks-news-databricks-unveils-mosaic-ai-agent-framework/>
 25. Create an AI agent - Databricks Documentation, accessed on March 18, 2025,
<https://docs.databricks.com/aws/en/generative-ai/agent-framework/create-agent>
 26. Unstructured retrieval AI agent tools - Azure Databricks | Microsoft Learn, accessed on March 18, 2025,
<https://learn.microsoft.com/en-us/azure/databricks/generative-ai/agent-framework/unstructured-retrieval-tools>
 27. Agentic RAG Framework and Databricks Mosaic AI | by Sandip Roy - Medium, accessed on March 18, 2025,
<https://roysandip.medium.com/agentic-rag-framework-and-databricks-mosaic-ai-e037c8323a51>
 28. Introduction to building gen AI apps on Databricks, accessed on March 18, 2025,
<https://docs.databricks.com/aws/en/generative-ai/agent-framework/build-genai-apps>
 29. Mosaic AI capabilities for generative AI apps - Databricks Documentation, accessed on March 18, 2025,
<https://docs.databricks.com/aws/en/generative-ai/guide/mosaic-ai-gen-ai-capabilities>
 30. Care Cost Compass: An Agent System Using Mosaic AI Agent Framework | Databricks Blog, accessed on March 18, 2025,
<https://www.databricks.com/blog/care-cost-compass-agent-system-using-mosaic-ai-agent-framework>
 31. Agents: Mosaic AI, Tools, and Function Calling | by AI on Databricks | Jan, 2025 - Medium, accessed on March 18, 2025,
<https://medium.com/@AI-on-Databricks/agents-mosaic-ai-tools-and-function-calling-2530c841f80c>
 32. CrewAI: A Guide With Examples of Multi AI Agent Systems - DataCamp, accessed on March 18, 2025, <https://www.datacamp.com/tutorial/crew-ai>
 33. Build agentic AI solutions with DeepSeek-R1, CrewAI, and Amazon SageMaker AI - AWS, accessed on March 18, 2025,
<https://aws.amazon.com/blogs/machine-learning/build-agentic-ai-solutions-with-deepseek-r1-crewai-and-amazon-sagemaker-ai/>
 34. Design multi-agent orchestration with reasoning using Amazon Bedrock and open source frameworks | AWS Machine Learning Blog, accessed on March 18, 2025,
<https://aws.amazon.com/blogs/machine-learning/design-multi-agent-orchestration-with-reasoning-using-amazon-bedrock-and-open-source-frameworks/>
 35. Revolutionizing AWS Security with AI Agents and CrewAI - Paul M. Duvall, accessed on March 18, 2025,
<https://www.paulmduvall.com/ai-agents-for-aws-security-businesses/>

36. Demo: Creating and Orchestrating AI Agents with CrewAI in AWS SageMaker - YouTube, accessed on March 18, 2025, <https://www.youtube.com/watch?v=YybbRP8HouQ>
37. Connect to any LLM - CrewAI, accessed on March 18, 2025, <https://docs.crewai.com/how-to/llm-connections>
38. CrewAI - Unity Catalog, accessed on March 18, 2025, <https://docs.unitycatalog.io/ai/integrations/crewai/>
39. Generative AI app developer workflow - Databricks Documentation, accessed on March 18, 2025, <https://docs.databricks.com/aws/en/generative-ai/tutorials/ai-cookbook/genai-developer-workflow>
40. CrewAI new features. Agentic AI framework | by Xin Cheng | Feb, 2025 - Medium, accessed on March 18, 2025, <https://billtcheng2013.medium.com/crewai-new-features-ff79f55cdc79>
41. Where can I get the latest crewAI documentation? - crewAI+ Help Center, accessed on March 18, 2025, <https://help.crewai.com/where-can-i-get-the-latest-crewai-documentation>
42. Crewai docs: comprehensive guide | Restackio, accessed on March 18, 2025, <https://www.restack.io/p/crewai-answer-comprehensive-guide-cat-ai>
43. Tools - CrewAI, accessed on March 18, 2025, <https://docs.crewai.com/concepts/tools>
44. Code Docs RAG Search - CrewAI, accessed on March 18, 2025, <https://docs.crewai.com/tools/codedocssearchtool>
45. Discussing CrewAI - Reddit, accessed on March 18, 2025, https://www.reddit.com/r/crewai/comments/1ixald2/discussing_crewai/
46. Welcome to CrewAI! - General, accessed on March 18, 2025, <https://community.crewai.com/t/welcome-to-crewai/5>
47. CrewAI Community Support, accessed on March 18, 2025, <https://community.crewai.com/c/community-support/7>
48. Discussions - crewAI Inc crewAI - GitHub, accessed on March 18, 2025, <https://github.com/crewAIInc/crewAI/discussions>
49. CrewAI, accessed on March 18, 2025, <https://www.crewai.com/>
50. Top 5 Open Source Frameworks for building AI Agents: Code + Examples - Reddit, accessed on March 18, 2025, https://www.reddit.com/r/LLMDevs/comments/1io0gnz/top_5_open_source_frameworks_for_building_ai/
51. 7 Awesome Platforms & Frameworks for Building AI Agents (Open-Source & More), accessed on March 18, 2025, <https://www.helicone.ai/blog/ai-agent-builders>
52. How to orchestrate Agents on Databricks using LangChain / LangGraph / OpenAI and more, accessed on March 18, 2025, <https://www.youtube.com/watch?v=RFCG67vQNeE>
53. Multi-Agent Orchestrator framework - Open Source at AWS, accessed on March 18, 2025, <https://awslabs.github.io/multi-agent-orchestrator/>
54. AWS Multi-Agent Orchestrator: A Guide With Examples - DataCamp, accessed on

March 18, 2025,

<https://www.datacamp.com/tutorial/aws-multi-agent-orchestrator>